



Tema 3: Librerías/Componentes de Spark

Rafael Pastor Vargas
Andrés Duque Fernández
SCC-ETSI Informática-UNED

LIBRERÍAS/COMPONENTES DE SPARK

- Librerías disponibles en Spark.
- Gestión de datos al modo SQL: Spark SQL.
- Machine Learning en Spark: Spark MLlib.
- **Procesamiento de gráfos: Spark GraphX.**
- Introducción a Spark Streaming.

TIPOS DE LIBRERÍAS

Graph algorithms







Standard & custom algorithms
Optimized for batch processing

Graph queries







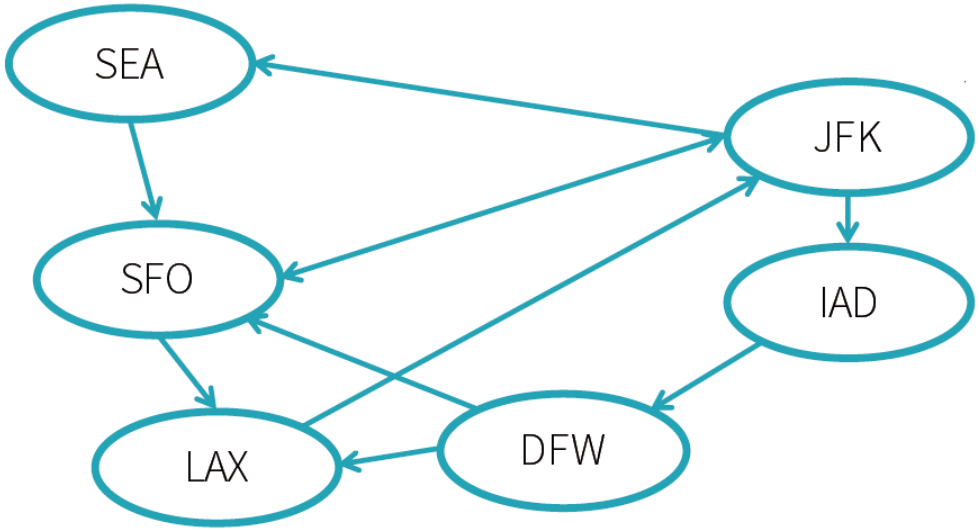
Motif finding
Point queries & updates

GraphFrames: Both algorithms & queries (but not point updates)

SPARK GRAPHFRAMES/GRAPHX

Arista (Edge)

src	dst	delay	tripID
"JFK"	"SEA"	45	1058923



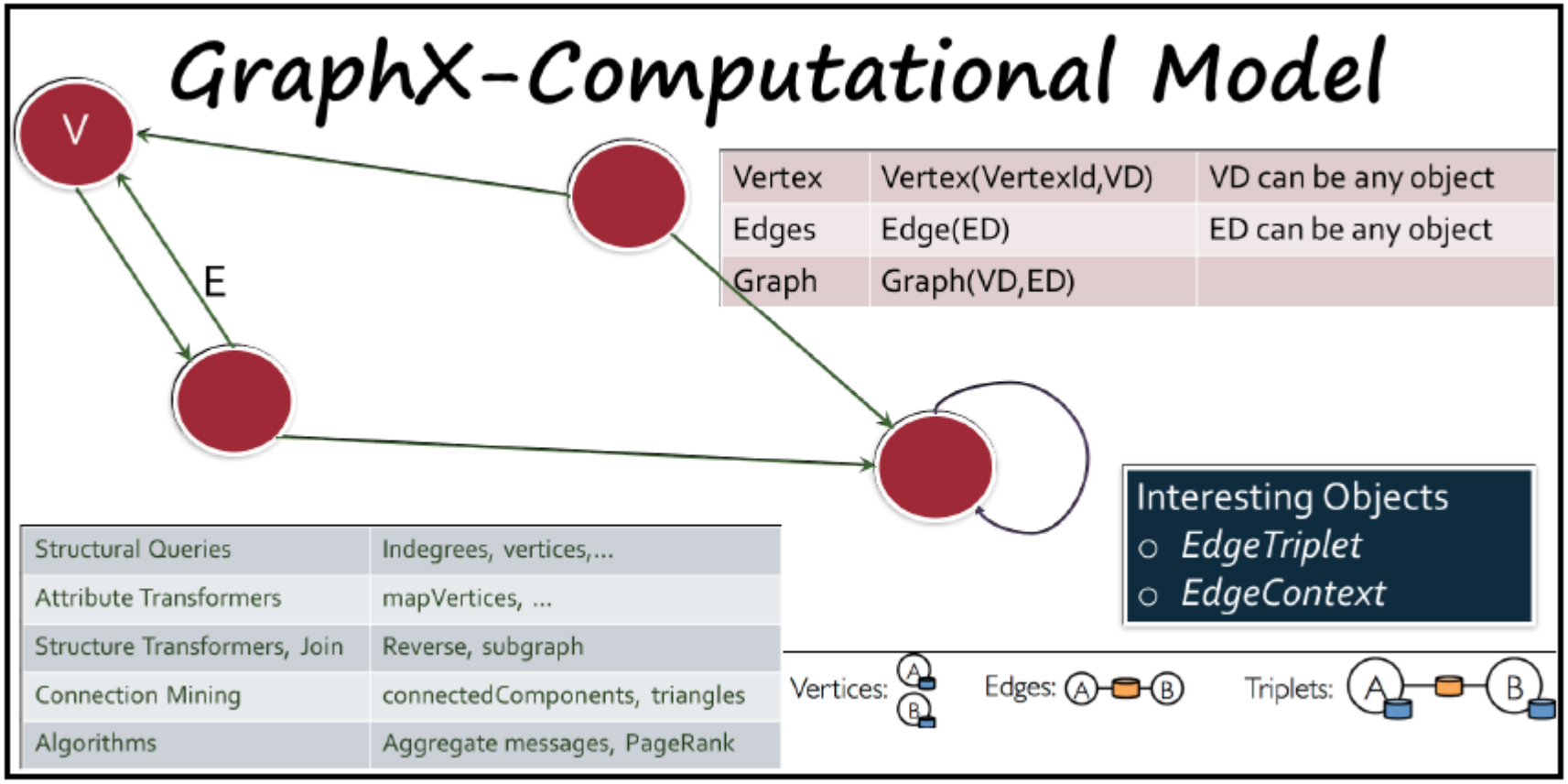
Vértice (Vertex)

id	City	State
"JFK"	"New York"	NY

GRAPHX

- Librería general de procesamiento de grafos
- Optimizada para computación distribuida eficiente (rapidez)
- Librería de algoritmos
  - PageRank, Connected Components, Triangle Sum, etc.
- No hay APIs para Java/Python
- Se usan los RDDs (API de bajo nivel)
- No utiliza las optimizaciones de spark: Catalyst/Tungsten

GRAPHX: MODELO COMPUTACIONAL



CREACIÓN DE GRAFOS

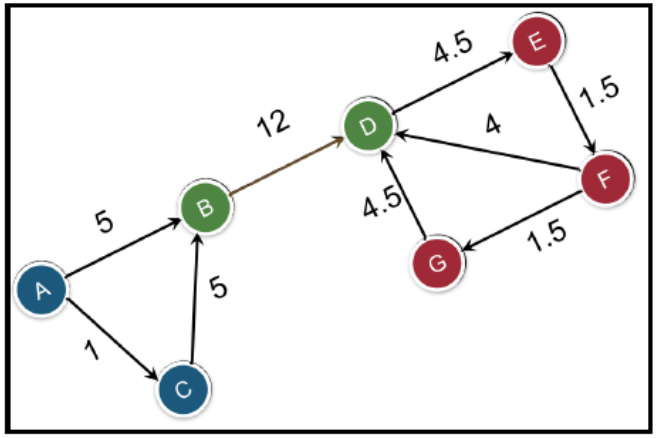
```
case class Person(name:String,age:Int)

val defaultPerson = Person("NA",0)

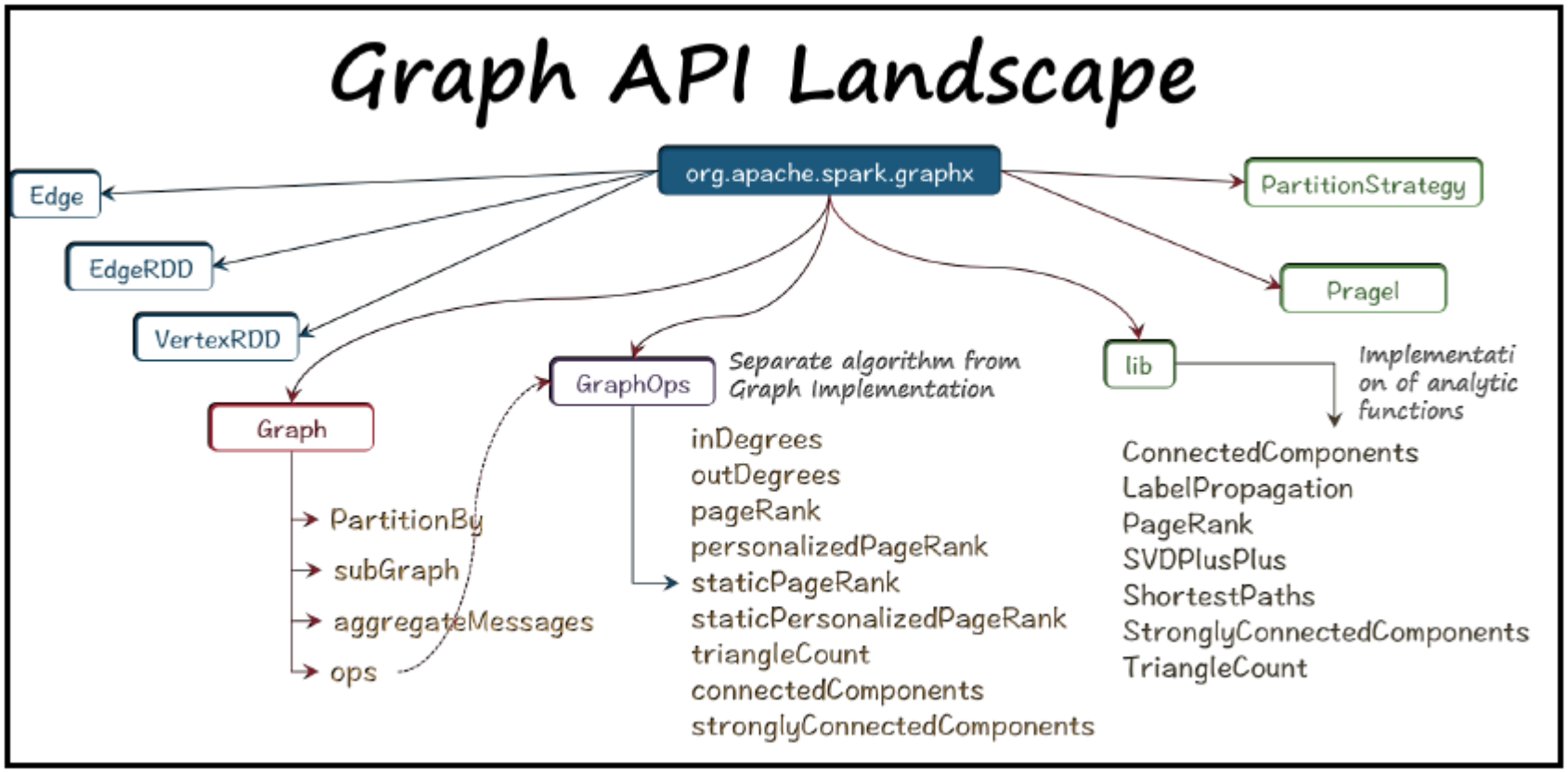
val vertexList = List( (1L, Person("Alice", 18)), (2L, Person("Bernie",
17)), (3L, Person("Cruz", 15)), (4L, Person("Donald", 12)), (5L,
Person("Ed", 15)), (6L, Person("Fran", 10)), (7L, Person("Genghis",854)) )

val edgeList = List( Edge(1L, 2L, 5), Edge(1L, 3L, 1), Edge(3L, 2L, 5),
Edge(2L, 4L, 12), Edge(4L, 5L, 4), Edge(5L, 6L, 2), Edge(6L, 7L, 2),
Edge(7L, 4L, 5), Edge(6L, 4L, 4) )
```

```
val vertexRDD = sc.parallelize(vertexList)
val edgeRDD = sc.parallelize(edgeList)
val graph = Graph(vertexRDD, edgeRDD,defaultPerson)
```



ESTRUCTURA DEL API



GRAPHX VERSUS GRAPHFRAMES

	GraphFrames	GraphX
Built on	DataFrames	RDDs
Languages	Scala, Java, Python	Scala
Use cases	Queries & algorithms	Algorithms
Vertex IDs	Any type (in Catalyst)	Long
Vertex/edge attributes	Any number of DataFrame columns	Any type (VD, ED)
Return types	GraphFrame or DataFrame	Graph[VD, ED], or RDD[Long, VD]

https://catalog.data.gov/dataset/airline-on-time-performance-and-causes-of-flight-delays-on-time-data

GRAPHFRAMES

- Dataframes: "vertices" & "edges"
  - Un vértice por fila; es necesario una columna id (ID único, clave)
  - Una arista por fila; columnas src,dst (usando ids de vertices.id)
  - El resto de columnas son propiedades(atributos) del vértice/arista

id	City	State
"JFK"	"New York"	NY
"SEA"	"Seattle"	WA

src	dst	delay	tripID
"JFK"	"SEA"	45	1058923
"DFW"	"SFO"	-7	4100224

GRAPHFRAMES: EJERCICIO

- Datos:
  - https://openflights.org/data.html
  - https://catalog.data.gov/dataset/airline-on-time-performance-and-causes-of-flight-delays-on-time-data
- Pyspark debe usar los paquetes de Graphframes
  - pyspark **--packages graphframes:graphframes:0.7.0-spark2.4-s_2.11**
  - graphframes.ipynb
- Ejemplo completo
  - https://databricks.com/blog/2016/03/16/on-time-flight-performance-with-graphframes-for-apache-spark.html

https://graphframes.github.io/graphframes/docs/_site/user-guide.html

CREACIÓN DEL GRAFO (1/3)

In [1]:
```
# Load data
airports = spark.read.option("Header",True).option("InferSchema", True).csv("airports.csv")
trips_usa_nov2017 = spark.read.option("Header",True).option("InferSchema", True).csv("216420986_T_ONTIME.csv")
```

In [2]:
```
airports.printSchema()
airports.show(5)
```

```
root
 |-- id: integer (nullable = true)
 |-- name: string (nullable = true)
 |-- city: string (nullable = true)
 |-- country: string (nullable = true)
 |-- IATA: string (nullable = true)
 |-- ICAO: string (nullable = true)
 |-- latitude: double (nullable = true)
 |-- longitude: double (nullable = true)
 |-- altitude: integer (nullable = true)
 |-- timezone: string (nullable = true)
 |-- DST: string (nullable = true)
 |-- tz: string (nullable = true)
 |-- type: string (nullable = true)
 |-- source: string (nullable = true)

+---+--------------------+------------+----------------+----+----+------------------+------------------+--------+--------+---+-
------------------+-------+-----------+
| id|                name|        city|         country|IATA|ICAO|          latitude|         longitude|altitude|timezone|DST|
tz|   type|     source|
+---+--------------------+------------+----------------+----+----+------------------+------------------+--------+--------+---+-
------------------+-------+-----------+
|  1|      Goroka Airport|      Goroka|Papua New Guinea| GKA|AYGA|-6.081689834590001|     145.391998291|    5282|      10|  U|P
acific/Port_Moresby|airport|OurAirports|
|  2|      Madang Airport|      Madang|Papua New Guinea| MAG|AYMD|    -5.20707988739|     145.789001465|      20|      10|  U|P
acific/Port_Moresby|airport|OurAirports|
|  3|Mount Hagen Kagam...| Mount Hagen|Papua New Guinea| HGU|AYMH|-5.826789855957031|144.29600524902344|    5388|      10|  U|P
acific/Port_Moresby|airport|OurAirports|
|  4|      Nadzab Airport|      Nadzab|Papua New Guinea| LAE|AYNZ|         -6.569803|        146.725977|     239|      10|  U|P
acific/Port_Moresby|airport|OurAirports|
|  5|Port Moresby Jack...|Port Moresby|Papua New Guinea| POM|AYPY|-9.443380355834961|147.22000122070312|     146|      10|  U|P
acific/Port_Moresby|airport|OurAirports|
+---+--------------------+------------+----------------+----+----+------------------+------------------+--------+--------+---+-
------------------+-------+-----------+
only showing top 5 rows
```

CREACIÓN DEL GRAFO (2/3)

In [3]:
```
trips_usa_nov2017.printSchema()
trips_usa_nov2017 = trips_usa_nov2017.drop("_c11")
trips_usa_nov2017.show(5)
```

```
root
 |-- ORIGIN: string (nullable = true)
 |-- ORIGIN_CITY_NAME: string (nullable = true)
 |-- ORIGIN_STATE_ABR: string (nullable = true)
 |-- ORIGIN_STATE_NM: string (nullable = true)
 |-- DEST: string (nullable = true)
 |-- DEST_CITY_NAME: string (nullable = true)
 |-- DEST_STATE_ABR: string (nullable = true)
 |-- DEST_STATE_NM: string (nullable = true)
 |-- DEP_DELAY: double (nullable = true)
 |-- ARR_DELAY: double (nullable = true)
 |-- CANCELLED: double (nullable = true)
 |-- _c11: string (nullable = true)

+------+--------------------+----------------+---------------+----+--------------+--------------+-------------+---------+------
---+---------+
|ORIGIN|    ORIGIN_CITY_NAME|ORIGIN_STATE_ABR|ORIGIN_STATE_NM|DEST|DEST_CITY_NAME|DEST_STATE_ABR|DEST_STATE_NM|DEP_DELAY|ARR_DE
LAY|CANCELLED|
+------+--------------------+----------------+---------------+----+--------------+--------------+-------------+---------+------
---+---------+
|   ABE|Allentown/Bethleh...|              PA|   Pennsylvania| DTW|   Detroit, MI|            MI|     Michigan|     -4.0|    -1
5.0|      0.0|
|   ABE|Allentown/Bethleh...|              PA|   Pennsylvania| DTW|   Detroit, MI|            MI|     Michigan|     -5.0|
-5.0|      0.0|
|   ABE|Allentown/Bethleh...|              PA|   Pennsylvania| DTW|   Detroit, MI|            MI|     Michigan|     -9.0|
-3.0|      0.0|
|   ABE|Allentown/Bethleh...|              PA|   Pennsylvania| ORD|   Chicago, IL|            IL|     Illinois|     -1.0|
8.0|      0.0|
|   ABE|Allentown/Bethleh...|              PA|   Pennsylvania| DTW|   Detroit, MI|            MI|     Michigan|     -6.0|
-4.0|      0.0|
+------+--------------------+----------------+---------------+----+--------------+--------------+-------------+---------+------
---+---------+
only showing top 5 rows
```

CREACIÓN DEL GRAFO (3/3)

- Para crear el grafo solo hay que proporcionar los dos dataframes (vertex, edges)
  - Adecuar los nombres de las columnas!!!

```
from graphframes import *
tripVertices = airports.withColumnRenamed("IATA", "id").distinct()
tripEdges = trips_usa_nov2017.select("DEP_DELAY", "ORIGIN", "DEST", "DEST_CITY_NAME", "DEST_STATE_NM")\
    .withColumnRenamed("ORIGIN","src").withColumnRenamed("DEST","dst")

# Generamos el grafo
tripGraph = GraphFrame(tripVertices, tripEdges)
```

OPERACIONES: CONSULTAS

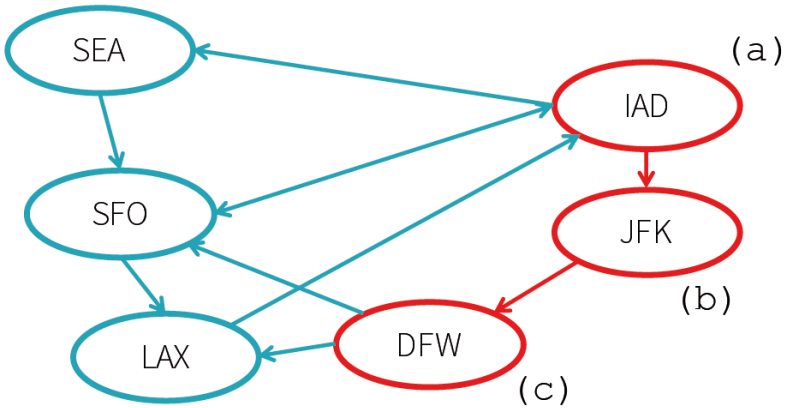
- Consultas SQL sobre vertices & aristas
  - Ejemplo: ¿Que viajes tienen un retardo significativo?
  - Ejemplo: ¿Cuantos aeropuertos hay en el estado de new York registrados?

- Consultas sobre los grafos
  - Grados del vertice
    - # aristas por vertice (entrantes, salientes, totales)
  - Tripletas

OPERACIONES: MOTIF

- Se usa para hacer búsquedas de patrones estructurales en el grafo

patterns = graghp.find(“ **(a)-[e1]->(b); (b)-[e2]->(c); (c)-[]->(a)** ”)



Filtro → patterns.filter(e1.DEP_DELAY > 60)

https://graphframes.github.io/graphframes/docs/_site/user-guide.html#motif-finding

ALGORTIMOS

- Encontrar los vértices importantes
  - PageRank
- Encontrar caminos entre conjuntos de vértices
  - Breadth-first search (BFS)
  - Shortest paths
- Encontrar grupos de vértices (componentes, intereses, comunidades)
  - Connected components
  - Strongly connected components
  - Label Propagation Algorithm (LPA)
- Otros
  - Triangle counting
  - SVDPlusPlus